Multi-Agent Architectures for LLM-Based Code Generation: An Empirical Comparison of Orchestration Strategies

Ivan Necerini s345147

Jacopo Rialti s346357

Emanuele Romano s346325

Marco Donatucci s337624

Ferdinando Del Vecchio s347426

Abstract

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities in code generation, enabling developers to produce functional software from natural language specifications. However, single-prompt interactions often fail on complex development tasks that require multi-step reasoning, iterative refinement, or careful handling of edge cases. This limitation has motivated the exploration of multi-agent architectures that decompose the software development process into specialized roles, potentially improving both code quality and functional correctness.

Multi-agent systems for code generation typically assign distinct responsibilities to different agents—such as planning, implementation, testing, and review—mirroring established software engineering practices. While this approach is conceptually appealing, the empirical benefits of such architectures remain underexplored. Key questions include whether the overhead of agent coordination is justified by quality improvements, whether specialized models for each role outperform a single model serving all roles, and whether adaptive strategies can reduce computational costs without sacrificing correctness.

This work presents a systematic empirical comparison of LLM-based code generation architectures, ranging from single-agent baselines to multi-agent pipelines with adaptive model routing. We implement and evaluate three primary architectures: (A) a single-agent baseline, (B) a multi-agent pipeline using a single model for all roles, and (C) a multi-agent pipeline with specialized models per role and adaptive routing based on task difficulty estimation. Additionally, we investigate the effect of prompt repetition, a recently proposed technique for improving non-reasoning LLMs (Leviathan et al., 2025), on code generation accuracy across all architectures.

Our evaluation is conducted on the HumanEval benchmark (Chen et al., 2021), comprising 164 hand-written programming problems with assertion-based tests. We address the following research questions:

- **RQ1**: Does a multi-agent pipeline with role separation (Planner, Developer, Tester, Reviewer) improve functional correctness compared to a single-agent approach?
- **RQ2**: Do specialized models assigned to each role provide measurable benefits over using a single model for all roles?
- **RQ3**: Can adaptive routing—selecting developer model capacity based on estimated task difficulty—reduce computational cost while maintaining or improving quality?
- **RQ4**: Does prompt repetition improve code generation accuracy for the models used in this study?

The contributions of this work include: (1) a reproducible multi-agent framework for code generation using LangGraph, (2) an empirical comparison of single-agent versus multi-agent architectures on standardized benchmarks, (3) an analysis of adaptive model routing based on Scrum-style story point estimation, and (4) an evaluation of prompt repetition effects in the context of code generation.

2 System Overview

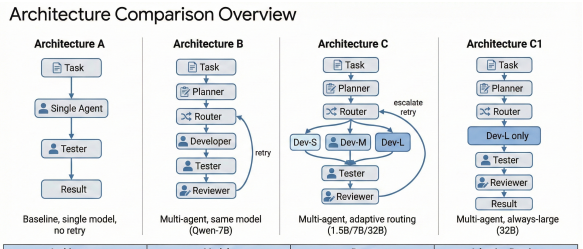
This section presents the architecture of our multi-agent code generation system, describing the logical pipeline, difficulty estimation mechanism, escalation policy, and the experimental configurations evaluated in this study.

2.1 Multi-Agent Pipeline Architecture

The proposed multi-agent pipeline decomposes the code generation task into five specialized components, each responsible for a distinct phase of the development process. The pipeline follows the sequence: **Planner** → **Router** → **Developer** → **Tester** → **Reviewer**, with conditional looping upon test failure.

The **Planner** agent receives the task specification and estimates its difficulty using Scrum-style story points, providing a rationale for the assessment. The **Router** uses this difficulty estimate to select an appropriately-sized Developer model, balancing computational cost against task complexity. The **Developer** agent generates the implementation code based on the task description and, in retry iterations, incorporates feedback from previous failures. The **Tester** is a non-LLM component that executes the generated code against assertion-based tests in a sandboxed environment. Finally, the **Reviewer** agent analyzes the test results and provides actionable feedback for subsequent iterations without modifying the code directly.

Figure 1 illustrates the four architectural configurations evaluated in this study, highlighting the differences in pipeline structure and model allocation.



Architecture	Models	Retry	Adaptive Routing
A	1	No	No
B	1	Yes	Simulated
C	3	Yes	Yes
C1	1 (large)	No	No

Figure 1: Comparison of the four experimental architectures: (A) single-agent baseline, (B) multi-agent with single model, (C) multi-agent with adaptive routing across three model tiers, and (C1) multi-agent always using the largest model.

2.2 Difficulty Estimation via Story Points

Task difficulty estimation is performed by the Planner agent using a Fibonacci-based story point scale: {1, 2, 3, 5, 8}. This scale, borrowed from Agile software development practices, serves as a routing signal rather than a time estimate. The Planner evaluates each task along five dimensions: (1) algorithmic complexity, (2) edge case density, (3) external constraints such as performance requirements or strict I/O formatting, (4) coupling with other components, and (5) specification ambiguity.

Story points are mapped to developer tiers as follows: points 1–2 indicate trivial to small tasks routed to the smallest model (Tier S), points 3–5 represent medium to challenging tasks routed to a mid-sized model (Tier M), and point 8 denotes hard tasks requiring the largest model (Tier L). This mapping is visualized in Figure 2.

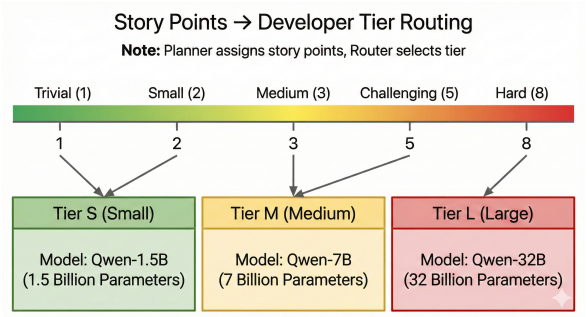


Figure 2: Mapping from story points to developer tiers. Each tier corresponds to a different model capacity: Tier S (1.5B parameters), Tier M (7B parameters), and Tier L (32B parameters).

2.3 Adaptive Routing and Escalation Policy

The system implements an adaptive escalation policy that progressively increases model capacity upon test failure. When the Tester reports a failure, the Router escalates the task to the next tier: S → M → L. A maximum of two escalations is permitted, after which the pipeline terminates regardless of test outcome.

Upon escalation, the Developer receives the previous code, failure history (error messages from test execution), and feedback from the Reviewer. This accumulated context enables the stronger model to address specific deficiencies identified in prior attempts. Figure 3 illustrates the escalation flow and termination conditions.

2.4 Experimental Architectures

We evaluate five architectural configurations to isolate the effects of multi-agent orchestration, model specialization, and adaptive routing:

Architecture A serves as the single-agent baseline. A single LLM receives the task and generates code in one pass without planning, review, or retry mechanisms.

Architecture B introduces the full multi-agent pipeline while keeping the model constant.

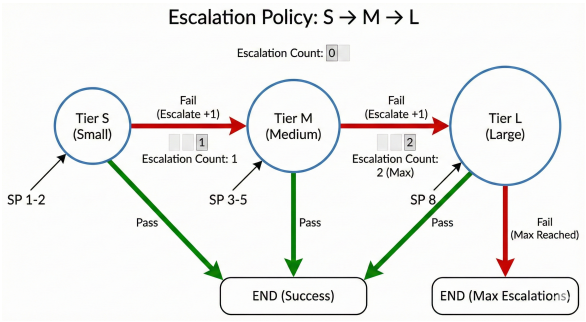


Figure 3: Escalation policy flow. On test failure, the system escalates from Tier S to M to L. The pipeline terminates upon test success or after reaching maximum escalations.

All roles—Planner, Developer (all tiers), and Reviewer—use the same model. This configuration isolates the effect of role separation and iterative refinement from model capacity differences.

Architecture C employs specialized models for each role. The Planner and Reviewer use a generalist model suited for reasoning and analysis. The Developer role uses capacity-matched models: a small model (1.5B) for Tier S, a medium model (7B) for Tier M, and a large model (32B) for Tier L.

Architecture C1 is a variant of C that bypasses adaptive routing by always using the Tier L developer (32B model), regardless of the Planner's difficulty estimate. This configuration serves as an upper-bound reference for comparing the efficiency-quality trade-off of adaptive routing.

Architecture C2 is an ablation study that removes the Tier S developer entirely. Tasks that would normally route to Tier S are instead handled by Tier M from the start. This configuration tests whether the smallest model tier is beneficial or introduces unnecessary failure cascades.

Table 1 summarizes the model assignments for each architecture.

Role	A	B	C	C1	C2
Planner	—	Q-7B	L-8B	L-8B	L-8B
Dev-S	—	Q-7B	Q-1.5B	Q-32B	Q-7B
Dev-M	—	Q-7B	Q-7B	Q-32B	Q-7B
Dev-L	—	Q-7B	Q-32B	Q-32B	Q-32B
Reviewer	—	Q-7B	L-8B	L-8B	L-8B
Baseline	Q-7B	—	—	—	—

Table 1: Model assignments per architecture. Q = Qwen2.5-Coder-Instruct, L = Meta-Llama-3-Instruct. Numbers indicate model size in billions of parameters.

2.5 Prompt Repetition Technique

We investigate the effect of prompt repetition, a technique recently proposed by Leviathan et al. (Leviathan et al., 2025) for improving non-reasoning LLM performance. The technique transforms the user prompt from ⟨QUERY⟩ to ⟨QUERY⟩\n\n⟨QUERY⟩, duplicating the input with a newline separator.

The rationale is that LLMs are causal language models where each token can only attend to preceding tokens. By repeating the prompt, tokens in the second occurrence can attend to all tokens in the first occurrence, potentially improving comprehension of complex instructions. The overhead is approximately 2× input tokens during the prefill stage, with no change to output token count. Figure 4 visualizes this attention pattern difference.

For each base architecture (A, B, C), we evaluate a corresponding prompt repetition variant (A-PR, B-PR, C-PR), applying repetition to all user-facing prompts while leaving system prompts unchanged.

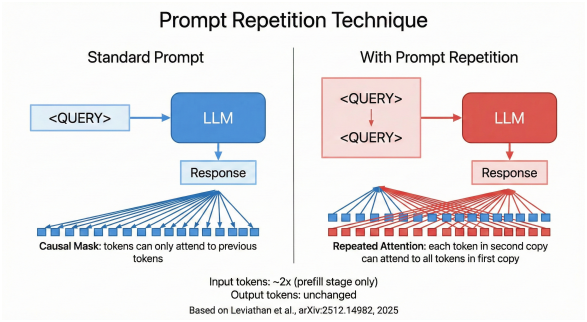


Figure 4: Comparison of attention patterns between standard prompting and prompt repetition. With repetition, tokens in the second copy can attend to all tokens in the first copy.

2.6 Implementation Details

The multi-agent pipeline is implemented using LangGraph, a framework for building stateful, graph-based LLM applications. The workflow is defined as a `StateGraph` where each agent corresponds to a node, and edges define the execution flow including conditional transitions based on test outcomes.

State management is handled through a `TypedDict`-based `GraphState` object that propagates task metadata, planning outputs, generated code, test results, and failure history across nodes. This structured state enables clean separation of concerns and facilitates debugging.

For execution monitoring and debugging, we integrate LangSmith, which provides tracing capabilities for inspecting individual LLM calls, token usage, and execution timelines. This tooling proved essential for diagnosing failure modes and optimizing prompt designs during development.

3 Experimental Setup

This section describes the dataset, execution environment, evaluation metrics, and experimental configurations used in our study.

3.1 Dataset

We evaluate all architectures on the HumanEval benchmark (Chen et al., 2021), a widely-used dataset for assessing code generation capabilities of large language models. HumanEval consists of 164 hand-written Python programming problems, each comprising a function signature with docstring, a set of assertion-based tests, and a reference solution (not used during generation).

Each task is structured as follows: the `prompt` field contains the function signature and docstring describing the expected behavior; the `test` field contains Python assertions that validate the implementation; and the `entry_point` field specifies the function name to be implemented. This structure enables automated evaluation through assertion-based testing, where generated code is concatenated with test assertions and executed in a sandboxed subprocess with a timeout of 5 seconds.

HumanEval was chosen for several reasons: it provides a controlled benchmark with diverse problem types ranging from simple string manipulation to algorithmic challenges; it uses assertion-based testing which yields unambiguous pass/fail outcomes; and it is widely adopted in the literature, facilitating comparison with prior work.

3.2 Execution Environment

All experiments were conducted on Kaggle Notebooks, a cloud-based computational platform providing reproducible execution environments. This choice ensures that experiments can be replicated without requiring local GPU resources.

Language models were accessed through the HuggingFace Inference API, which provides serverless endpoints for model inference. This approach offers several advantages: it eliminates the need for local model deployment, provides consistent inference performance, and enables access to models of varying sizes (1.5B to 32B parameters) without hardware constraints.

3.3 Evaluation Metrics

We employ four categories of metrics to comprehensively evaluate the architectures.

Primary Metrics. The primary measure of functional correctness is the **Pass Rate**, defined as the percentage of tasks where all test assertions pass. We also report **Pass@1**, which measures correctness on the first generation attempt—for Architecture A this equals the pass rate, while for multi-agent architectures it reflects success before any retry or escalation. To quantify uncertainty, we compute **95% confidence intervals** using the Wilson score interval for binomial proportions.

Cost Metrics. To assess computational efficiency, we measure **Total Tokens** (sum of input and output tokens per task), **API Calls** (number of LLM invocations per task), and **Execution Time** (end-to-end wall-clock time per task in seconds). These metrics enable cost-benefit analysis of multi-agent overhead.

Adaptive Metrics. For architectures employing the multi-agent pipeline (B, C, C1, C2), we track additional metrics specific to the adaptive mechanisms. **Retry Count** measures the number of Developer→Tester→Reviewer loops executed until success or final failure. **Escalation Count** tracks the number of tier transitions (S→M or M→L) triggered by test failures. **Story Point Accuracy** evaluates the correlation between the Planner's difficulty estimate and actual task outcome. **Tier Distribution** reports the percentage of tasks routed to each developer tier (S/M/L).

Static Code Quality Metrics. Beyond functional correctness, we assess the quality of generated code using static analysis. **Cyclomatic Complexity (CC)** measures the number of linearly independent paths through the code, computed using the Radon library. Lower values indicate simpler, more testable code (1–5: simple, 6–10: moderate, 11+: complex). **Maintainability Index (MI)** is a composite metric incorporating Halstead Volume, cyclomatic complexity, and lines of code, with values ranging from 0 to 100 (higher indicates better maintainability). These metrics are computed for all generated solutions, enabling analysis of whether multi-agent pipelines produce more maintainable code.

3.4 Experimental Configuration

We evaluate eight architectural configurations, as summarized in Table 1. Architectures A and A-PR represent single-agent baselines with and without prompt repetition. Architectures B and B-PR employ the full multi-agent pipeline with a single model for all roles. Architectures C and C-PR use specialized models with adaptive routing based on story points. Architecture C1 always uses the largest developer model regardless of difficulty estimate, serving as an upper-bound reference. Architecture C2 is an ablation study that removes the smallest developer tier.

Each configuration was evaluated on all 164 HumanEval tasks. For multi-agent architectures, a maximum of two escalations was permitted before terminating the pipeline. All prompt repetition variants (A-PR, B-PR, C-PR) apply the repetition technique to user prompts only, leaving system prompts unchanged.

4 Experimental Results

5 Conclusion

References

Mark Chen and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Yaniv Leviathan and 1 others. 2025. Prompt repetition improves non-reasoning llms. *arXiv preprint arXiv:2512.14982*.